Hierarchical User Memory with Backward Consolidation for POI Recommendation

Anonymous Author(s)

Abstract

Place recommendation needs both fine-grained behavioral signals and a structured memory that can be updated as preferences evolve. Recent memory-augmented LLM agents [3, 5, 6] show that structured memory compresses long histories without degrading downstream quality. We propose a **POI-centric Hierarchical Note-based Memory** that turns raw POI interaction logs into a three-level tree grounded in a POI taxonomy and maintained dynamically. Building on PersonaMem-v2 [3], the system adds three POI-specific pieces: dynamic LLM-driven routing, a backward consolidation pass (merge, elevate, prune), and three-axis preference separation (demographics, explicit, implicit).

Our central finding, on a 4-way POI MCQ over N=500 users, is that *the wording of the elevation prompt matters more than the structural elevation choice itself.* Two naive prompts that change the structure (one rewrites children, *destructive*, 0.740; one does not, *additive*, 0.724) both fall below the strongest baseline FixedHier (0.760). Adding a one-line **concrete-noun guard**, a rule that forbids replacing subcategory words such as "Korean" with abstract ones such as "Asian cuisine," restores accuracy to 0.758, matching FixedHier within the ±0.038 Wilson 95% CI.

At this accuracy, the system is also the most compact memory among the deployable systems we evaluated: 2,621 tokens per user, 28× smaller than the full-context oracle and 12% smaller than FixedHier. The compression decomposes into three independent axes against FixedHier (fewer notes per user, shorter notes, and a smaller total footprint), which we summarize in a single Memory Efficiency Score (MES, defined in Section 4.2) on which Ours ranks first among all evaluated configurations. The backward consolidation pass primarily serves as a compression mechanism: removing it inflates memory by 16% with no measurable accuracy gain. For deployed POI memory, the elevation prompt is the primary design choice; structural choices such as dynamic vs. fixed hierarchy contribute less to accuracy.

CCS Concepts

• **Information systems** → **Recommender systems**; • **Computing methodologies** → *Natural language processing*; *Knowledge representation and reasoning*.

Keywords

personalization, Point-of-Interest (POI) recommendation, memory-augmented LLM agents, hierarchical user memory

1 Introduction

Relevance to the Web. Place search and recommendation on web map services run on exactly this kind of user model; this paper studies how an LLM-maintained hierarchical memory for it should be consolidated, and which prompt-level design choices matter.

Personalization drives user satisfaction and click-through rate (CTR) in modern platform services. In POI (Point-of-Interest)-centric services such as place recommendation, accurate modeling of users' preferred place types and visitation patterns is critical. Prior work constructs user personas either by injecting the full interaction history into a large language model (LLM) [9] or by chunking and retrieving relevant pieces via similarity [20, 21]; the resulting personas serve as context in recommendation and search, and their quality directly affects downstream performance.

Generating high-quality personas from real-world POI logs is challenging. Logs (search, save, click, visit, reservation) are large-scale and noisy, with much exploratory behavior that does not reveal preferences. Place preferences also emerge gradually rather than from any single event. Explicit signals such as reviews are sparse and cover only a subset of users, while implicit behaviors (repeated clicks within a category, sustained exploration, prolonged dwell time) are pervasive [15]. Any single implicit signal is ambiguous, but consistent patterns across interactions reveal structured latent preferences.

Because preferences evolve, the system must reconcile historical patterns with current interests. These properties expose limitations of existing methods. Full-context approaches quickly hit context limits and suffer from lost-in-the-middle effects [10]. They score well on direct factual recall (92.85% on LoCoMo [9]) but lose to structured memory on implicit personalization (45.6% vs. 55.2% [3]), and at much higher cost. Retrieval-based chunking avoids the context limit but fragments the already-sparse POI signal. Flat chunks also cannot move freely between fine-grained categories (e.g., Korean, Japanese, cafés) and higher-level tendencies such as a dining-focused lifestyle.

Recent memory-augmented LLM agents take varied approaches. Mem0 [5] maintains a flat list with add/update/delete operations but lacks hierarchy. A-Mem [6] adds structured notes with cross-node activation but no explicit hierarchy management. PersonaMem-v2 [3] introduces a three-level tree with dual explicit/implicit extraction and outperforms full-context baselines, but relies on fixed predefined slots and performs no structural maintenance such as cross-level trait promotion. For POI specifically, Feng et al. [16] probe zero-shot next-POI ranking with LLMs but report that the models struggle with geographical reasoning and candidate ordering, highlighting the need for structured user-side memory rather than prompt-only inference.

We propose a POI-centric Hierarchical Note-based Memory and a Memory Agent that maintains it dynamically (Figure 1). The memory aggregates POI activities into topic-specific notes via LLM reasoning, consolidating dispersed implicit preferences into compact units, and organizes them in a three-level POI taxonomy. Preference information is stored at the appropriate abstraction level (e.g.,

WWW '27, Dublin, Ireland
2027. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

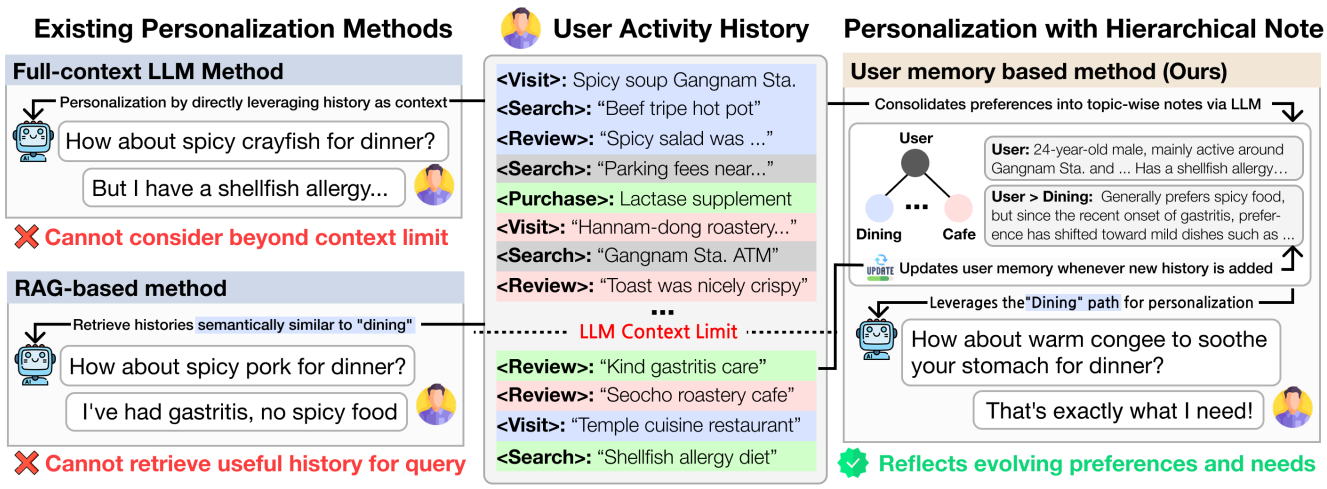


Figure 1: Personalization on a dining query for a user whose preferences have evolved (spicy → mild after gastritis). *Left:* a full-context LLM cannot reason beyond its context limit, and RAG retrieves entries semantically similar to "dining" but misses the gastritis signal, so both recommend spicy dishes. *Middle:* the raw user-activity history is long enough to exceed the LLM context window, and the gastritis-related entries are scattered across sessions. *Right:* our hierarchical-note memory consolidates activities into topic-wise notes (e.g., *Dining*, *Cafe*), updates the relevant note whenever new history is added, and leverages the *Dining* path at inference time to surface the evolved preference and recommend a context-appropriate "warm congee".

region-, category-, or situation-based), so downstream consumers can read only the slice they need.

Our contributions are:

- **C1 (system).** A POI-centric hierarchical note memory extending PersonaMem-v2 [3] with dynamic LLM-driven routing, three-axis preference separation (demographics, explicit, implicit), and a backward consolidation pass (merge, elevate, prune).
- **C2 (central finding).** On a 4-way POI MCQ over N=500 users, the elevation *prompt* matters more than the structural elevation choice: a one-line **concrete-noun guard** raises accuracy from 0.724–0.740 (naive variants) to 0.758, statistically tied with FixedHier (0.760) within the ±0.038 Wilson 95% CI.
- **C3 (three-axis compression at parity).** At the same accuracy, Ours compresses on three axes vs. FixedHier (notes/u −8.2%, tok/note −4.0%, tokens/u −11.8%) and is 28× smaller than the full-context oracle, topping the Memory Efficiency Score (MES = 0.149; Eq. 5).
- **C4 (backward pass as footprint lever).** Removing the backward pass moves accuracy within the CI (0.758 → 0.734) but inflates tokens by 16.4%: a compression mechanism, not an accuracy contributor.
- **C5 (evaluation).** A typed 4-way MCQ probe adapted from PersonaGym [12] (T1 recall, T2 cross-signal, T3 evolution) reporting per-type accuracy at N=500.

2 Related Work

Memory architectures. Building on LLM-agent frameworks [23–25], recent work on agent memory varies along two axes: structure (flat vs. hierarchical) and maintenance (append-only vs. actively managed). Mem0 [5] keeps an unstructured list with add/update/delete; MemGPT [8] extends context via virtual paging; neither imposes semantic organization. A-Mem [6] uses Zettelkasten-style notes with cross-node activation but no taxonomy-grounded hierarchy or backward maintenance. EMem [7] matches full-context quality on LoCoMo with ~32× fewer tokens via expectation-guided storage. MemoryBank [34] adds Ebbinghaus-curve forgetting, and HippoRAG [35] models cortex-hippocampus indexing with a knowledge graph; both show biologically inspired structure helps, but neither adapts topology dynamically. PersonaMem [4] and PersonaMem-v2 [3] are closest to our work: PersonaMem-v2 proposes a three-level tree with dual explicit/implicit extraction and outperforms full-context baselines (55.2% vs. 45.6% at 16× fewer tokens). We extend this paradigm with dynamic LLM-driven routing, common trait elevation as a backward maintenance operation, and a third extraction axis with root-exclusive routing.

Full-context vs. structured memory. Long-context LLMs lead on factual recall [9] but degrade on middle-position recall [10], needle-in-a-haystack [46], and RULER [42]. RAG [26, 27] sidesteps context limits but assumes verbatim answers in retrieved passages. For implicit preferences (signals distributed across many ambiguous behavioral events), structured memory is preferable. In our setting, implicit extraction averages 73K tokens per user, beyond practical limits at deployment scale; we therefore use full-context as a quality validation baseline, not a deployment option.

Recommendation and persona construction. Classical sequential recommenders [28, 29] and trajectory models [30, 38, 39] learn

dense representations but require large supervised signal with limited interpretability. LLM-based recommenders [31, 32, 40, 43, 45] unify recommendation as language processing. Our system shares the language-grounded representation with P5/TallRec but separates persona construction from recommendation: the persona is an interpretable, evolving artifact built offline and injected into downstream prompts. Persona generation has been studied from synthetic seeds [11], reward signals [14], and interaction histories [15]; evaluation methodology comes from PersonaGym [12], PersonaBench [13], and LaMP [36], with MCQ probing following MMLU [33]. Parameter-side personalization [37, 44] is complementary to our prompt-context approach. For POI specifically, Feng et al. [16] report zero-shot next-POI ranking with LLMs is sensitive to candidate ordering and limited in geographical reasoning. PersonaDB [17] and RMM [18] explore retrieval-based persona selection; we differ by maintaining a *persistent, evolving* memory with structural maintenance, accumulating evidence across sessions rather than one-shot extraction.

3 Proposed Method

3.1 Overview

Our framework constructs POI-centric user memory in three stages. **(1) Signal Extraction** converts raw POI interaction data (search queries, clicks, saves, reviews) into structured profiles, processing explicit signals (reviews) and implicit signals (behavioral logs) separately. **(2) Hierarchical Note Memory** stores extracted features in a taxonomy-grounded three-level tree, maintained by a **Memory Agent** that performs forward aggregation of new signals and backward consolidation for structural integrity. **(3) Downstream Consumption** reads relevant notes from the hierarchy and injects them into an LLM-as-reranker prompt [41] as the production target. In this paper we evaluate upstream memory quality via the MCQ probe in Section 4.1, isolating memory design from reranker-specific behavior.

3.2 POI Interaction Signal Extraction

User preferences for places appear through two different channels: explicit signals written by the user, and implicit signals inferred from behavior. We extract a structured preference profile from each channel separately, then merge them in the hierarchical memory. This dual-extraction approach follows the paradigm established by PersonaMem-v2 [3], adapted to the POI domain where implicit behavioral signals are particularly abundant. A third demographic axis is introduced later, during forward aggregation (Section 3.4), with root-exclusive routing.

3.2.1 Explicit Signal Extraction. Explicit signals originate from user-authored content (primarily place reviews), where preferences are stated directly in natural language (e.g., "I love spicy broth" or "the quiet atmosphere was perfect for working"). We process review histories with an LLM-based extraction pipeline that, for each user, produces a set of structured persona items. Each item consists of four fields: (a) *User_Info*, a single atomic fact about the user's lifestyle or preference distilled from the review evidence; (b) *Rationale*, a chain-of-thought justification grounded in specific review dates, place names, and quoted expressions; (c) *User_Vocabulary*, verbatim words and phrases from the user's original text to preserve linguistic identity; and (d) *Reference*, indices into the source review records.

To handle the wide variance in review volume across users, we employ two few-shot prompt strategies: a *rich* variant with detailed examples for users who have written many reviews, and a *sparse* variant with conservative, observation-oriented examples for users with limited review data. The sparse strategy deliberately avoids over-generalizing from insufficient evidence, instead emphasizing factual descriptions of observed behavior.

3.2.2 Implicit Signal Extraction. Implicit signals are derived from non-verbal behavioral traces: search queries, POI clicks, dwell patterns, and place saves. Unlike reviews, these actions do not directly articulate preferences, but consistent patterns across many interactions can reveal latent interests [15]. We construct a profiling pipeline that transforms raw click logs into structured personas through the following steps:

(1) **Sessionization.** Consecutive interactions involving the same query or the same POI within a 60-minute window are grouped into micro-sessions. Each session records the query text, timestamps, day of week, clicked POI name and category, location, review popularity rank, and click count.
(2) **Demographic context attachment.** Available demographic attributes (age, gender, marital status, household composition, occupation, and children information) are prepended to the session log as structured metadata. These attributes are later routed exclusively to the root node's `user_info` field during hierarchical path mapping (Section 3.4), enforcing a clean separation between demographic facts and behavioral preferences.
(3) **LLM-based persona profiling.** The sessionized log is passed to an LLM with strict constraints: (a) all persona claims must cite specific click evidence; (b) hallucination of unobserved behaviors (e.g., inferring residence from visit locations) is prohibited; (c) stereotypical reasoning based on demographics is forbidden; and (d) the output must decompose preferences into predefined POI domains (dining, cafés, travel, medical, beauty, shopping, etc.).

The output is a domain-partitioned persona string, where each domain section describes the user's inferred preferences with concrete behavioral evidence. To capture temporal dynamics, we apply this extraction over sliding windows (e.g., 21-day periods) across the observation period and aggregate the per-window results into a unified implicit persona per user.

3.3 Hierarchical Note Memory Structure

The core data structure is a three-level tree grounded in a predefined POI taxonomy, designed to support preference representation at multiple levels of abstraction. This structure extends the three-level paradigm of PersonaMem-v2 [3] with domain-specific customization for POI services and dynamic path management.

3.3.1 POI Taxonomy. The base hierarchy defines Level 1 as the root node (representing the user as a whole), Level 2 as broad POI domains, and Level 3 as fine-grained subcategories within each

domain. The predefined taxonomy includes 11 Level 2 domains—*Dining*, *Café/Dessert*, *Transportation/Mobility*, *Travel/Accommodation*, *Leisure/Sports*, *Culture/Arts*, *Self-development*, *Pets*, *Shopping*, *Medical*, and *Beauty*—each containing 2–10 Level 3 subcategories (e.g., Dining → {Korean, Japanese, Chinese, Western, Fast Food, ...}). Unlike the fixed-slot approach of PersonaMem-v2, the Memory Agent may dynamically extend this taxonomy by creating new Level 3 nodes when user data warrants categories not present in the base hierarchy, subject to the hierarchical consistency constraints described in Section 3.4. A maximum depth of three levels is enforced both at the prompt level and through structural validation.

3.3.2 Note Schema. Each node in the hierarchy is associated with a *Note*, a structured record that stores the accumulated preference knowledge for that category. A note contains the following fields:

- **path**: the full hierarchical path (e.g., "User > Dining > Korean Food"),
- **user_info**: demographic and factual attributes (populated *only* at the root node; e.g., "24-year-old male, lactose intolerant, frequently active near Gangnam Station"; all sub-nodes are constrained to empty lists),
- **summary**: a natural-language paragraph summarizing the user's preferences and behavioral patterns for this category, integrating both explicit and implicit signals (e.g., for the Korean Food node: "favors spicy and savory dishes such as spicy pork," inferred from repeated visits and supportive review excerpts),
- **keywords**: BM25-oriented keyword tags for retrieval,
- **updated_count**: the number of times the note content has been substantively modified,
- **recently_updated**: the date of the most recent content update (e.g., the Korean Food note advances from 2025/02/19 to 2025/02/25 after new activity arrives, while the Tennis note, last updated on 2024/10/23, remains unchanged).

As illustrated in Figure 2, a user's initial memory may contain nodes such as `User > Dining > Korean` and `User > Exercise > Tennis`, each holding independently tracked preference notes. When new activity arrives (for example, on 2025/02/25 a review mentioning gastritis care, a visit to Korean temple food, and a search for mild Chinese dishes), the forward pass selectively updates only the affected note (Korean), leaving the unrelated Tennis note untouched. The backward pass subsequently merges semantically overlapping siblings, elevates traits shared across siblings to their parent (preserving concrete subcategory words at the child), and prunes leaf nodes that have not been updated within a configurable staleness window.

All notes for a user are stored atomically in a single Parquet file, enabling efficient bulk I/O. The tree structure metadata (parent-child relationships, node levels) is maintained in a separate hierarchy file. This separation allows the Memory Agent to load, modify, and persist the entire memory state in a single read-modify-write cycle per operation.

3.4 Memory Agent: Forward Aggregation

When new interaction data arrives (either explicit persona extractions or implicit persona profiles), the Memory Agent executes a three-step forward pass to integrate the information into the existing memory. Each step is mediated by an LLM call; for a user with N distinct target paths identified in Step 2, the total cost is

$$\#\text{LLM calls}(u) = 2 + N(u), \tag{1}$$

where the constant 2 accounts for the single extraction call (Step 1) and the single path-mapping call (Step 2), and Step 3 contributes one call per target path.

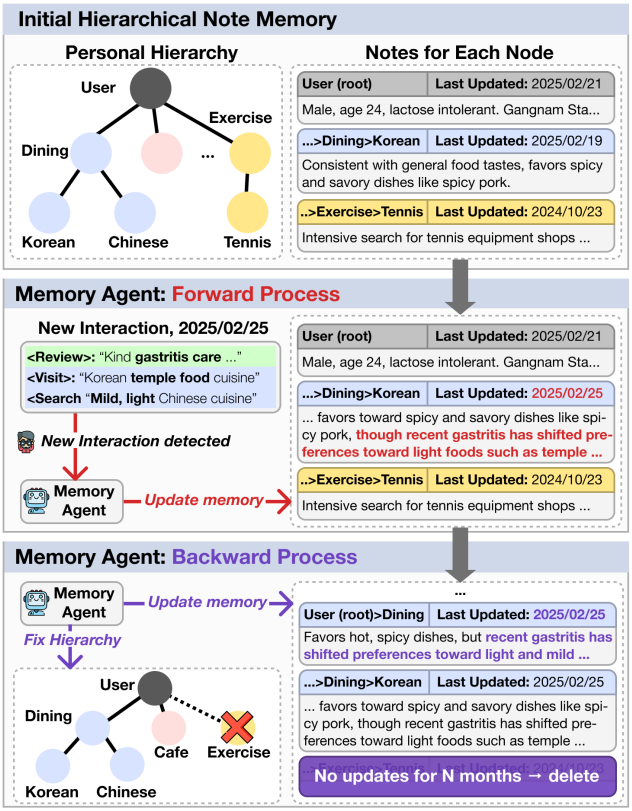


Figure 2: Memory Agent operation. *Initial*: per-node notes anchored to a personal hierarchy. *Forward*: a new interaction selectively updates only the affected note (Korean). *Backward*: a shared trait is elevated from child to parent with concrete subcategory words preserved at the child, and stale leaves are pruned.

3.4.1 Step 1: Preference Profile Extraction. The raw interaction text, combining both implicit behavioral profiles and explicit review extractions, is analyzed by an LLM to separate three categories of information:

- **User information** ($\mathcal{U}$): objective, slowly-changing facts such as age, occupation, and primary activity region.
- **Explicit preferences** ($\mathcal{P}_e$): stated preferences where the user directly expresses likes, dislikes, or requirements in language (e.g., "I prefer acidic coffee" or "I look for outlets at cafés").
- **Implicit preferences** ($\mathcal{P}_i$): latent preferences inferred from non-verbal behavioral patterns (e.g., "tends toward isolated,

productivity-oriented spaces" derived from repeated clicks on quiet study cafés).

This three-axis separation extends the two-axis approach of PersonaMem-v2 [3] (which combines demographics with explicit preferences) by isolating demographic facts into a dedicated channel. The LLM is instructed to preserve concrete keywords from the source data rather than abstracting them (e.g., extracting specific dish names rather than "diverse menu"), to merge semantically similar items into compact statements, and to strictly distinguish between exploratory actions (clicks, searches) and confirmed actions (visits, reservations) to avoid inflating behavioral claims.

3.4.2 Step 2: Hierarchical Path Mapping. Unlike supervised taxonomy expansion methods such as TaxoExpan [47], our routing is performed in-context by an LLM that jointly considers base-taxonomy paths and any user-specific paths created in earlier sessions. The mapping enforces five principles: *(i) root exclusivity for demographics—$\mathcal{U}$* items are mapped only to the root `user_info` field, never as subcategory nodes; *(ii) context-aware routing*—"prefers quiet atmosphere" goes to "User > Café" if surrounding evidence is café-related, otherwise to the root; *(iii) semantic reuse*—existing paths are preferred when they subsume the new item (a "latte" preference maps to "Coffee/Tea" rather than a new "Latte" sibling); *(iv) hierarchical consistency*—new Level-3 nodes must match the abstraction level of their siblings ("Korean"/"Japanese" under "Dining" yes; "Naengmyeon" no, it belongs under "Korean"), and nodes at Level 4 or deeper are prohibited; *(v) aggregation*—items targeting the same path are grouped into one update call.

3.4.3 Step 3: Note Update. For each target path, an LLM acts as an *editor* rather than appender [5]: it compares new data with existing content, resolves conflicts in favor of recency, removes outdated entries, and consolidates redundant statements. The root node manages demographics in its `user_info` field; sub-nodes are restricted to category-specific preferences. Substantive changes increment `updated_count` and refresh `recently_updated`.

3.5 Memory Agent: Backward Consolidation

Over time, repeated forward passes may introduce redundancy across sibling nodes, scatter shared preferences across leaf notes, or leave stale nodes from transient interests. The backward consolidation pass addresses these issues through three sequential operations, executed in a specific order: merging first reduces the number of siblings, which yields clearer common traits for the subsequent elevation step, and pruning last removes nodes that may have become stale after reorganization.

3.5.1 Sibling Node Merging. For each parent node with two or more children, the Memory Agent presents all child notes to an LLM and asks it to identify semantically overlapping siblings that should be merged. Only leaf nodes (those without children of their own) are eligible for merging. When a merge is approved, the LLM produces a unified `merged_note` that integrates the content of the source nodes, removing duplication while preserving unique details. The merged note inherits the cumulative `updated_count` and the most recent timestamp from its sources. Source nodes are then deleted from both the note store and the hierarchy. If only part of a source node fits the merge (the rest belonging to a different theme), the LLM also returns an `updated_source` holding the remaining content, which is written back to the original node.

3.5.2 Common Trait Elevation. After merging, the Memory Agent examines each parent–children group to identify preferences that appear across two or more sibling notes. An LLM extracts these common traits and rewrites the parent note to incorporate them, while simultaneously rewriting each child note to remove the elevated content.

This operation enforces a hierarchical invariant: shared preferences reside at the appropriate abstraction level, not duplicated across subcategories. Cross-subcategory tendencies (e.g., a general preference for spicy food) belong at the domain level ("Dining"), not replicated in "Korean," "Chinese," and "Asian" subcategory notes. The process respects level semantics: the root node absorbs user-wide commonalities, Level 2 nodes absorb domain-wide patterns, and Level 3 nodes retain only subcategory-specific details.

The over-abstraction failure mode. A naive elevation introduces a specific failure mode. When asked to summarize what is shared across "Korean," "Chinese," and "Japanese" notes, the LLM tends to replace these concrete words with an abstract superclass: writing "*enjoys Asian cuisines*" at the Dining parent and stripping "Korean," "Chinese," and "Japanese" from each child. The hierarchy is preserved in shape, but the persona loses the words a downstream reranker needs to distinguish a Korean-leaning user from a Japanese-leaning one.

Concrete-noun guard. We address this with a prompt-level rule that leaves the structural operation unchanged. The elevation prompt is given one extra instruction: *concrete category nouns appearing in child notes must be kept at either the parent or the child.* The LLM may use an abstract superclass at the parent only if it also keeps the concrete forms (e.g., "enjoys Asian cuisines (Korean, Chinese, Japanese)"). We compare this guard against two alternatives: the *destructive* variant (rewrites children to remove the elevated trait, no concrete-noun rule) and the *additive* variant (leaves children intact, accepting duplication). Section 4.3 shows this prompt-level choice produces a +0.034 accuracy gain and is the primary determinant of accuracy in the elevation step.

Formally, let $C(n)$ denote the set of traits stored at note n. Given a parent node p with sibling set $\mathrm{ch}(p)$, let $I = \{\, i : |\{n_k \in \mathrm{ch}(p) : i \in C(n_k)\}| \geq 2 \,\}$ be the set of traits appearing in two or more siblings. The elevation operation rewrites the parent and child contents as

$$\begin{aligned} C(p) &\;\rightarrow\; C(p) \cup I, \\ C(n_k) &\;\rightarrow\; C(n_k) \setminus I, \quad \forall\, n_k \in \mathrm{ch}(p). \end{aligned} \tag{2}$$

Let $S = \sum_n |C(n)|$ denote total stored content, and let $k_i = |\{n_k \in \mathrm{ch}(p) : i \in C(n_k)\}|$ denote how many siblings contain trait i (with $k_i \geq 2$ by definition of I). Assuming the elevated traits are not already at the parent ($I \cap C(p) = \emptyset$), the elevation reduces total content by

$$\Delta S \;=\; -\sum_{i \in I} (k_i - 1) \;\leq\; -|I|, \tag{3}$$

a hierarchical compression analogous to the Minimum Description Length principle [22]. To our knowledge, this explicit trait elevation operation has not been proposed in prior memory systems [3, 5–7].

3.5.3 Stale Leaf Pruning. Finally, the Memory Agent removes leaf nodes that represent transient or exploratory interests that did not develop into sustained preferences. A leaf node n is pruned if it satisfies two conditions simultaneously:

$$\text{updates}(n) \leq \tau_u \quad \text{and} \quad \text{age}(n) > \tau_d, \tag{4}$$

where $\text{updates}(n)$ is the cumulative update count of the node (the schema field `updated_count`), $\text{age}(n)$ is the time since its last update in days, τ_u is the minimum update threshold (default: 2), and τ_d is the staleness threshold in days (default: 120). Nodes are processed in decreasing depth order so that pruning a leaf may expose its parent as a new leaf, which is then evaluated in the same pass. This mechanism prevents the memory from accumulating noise from one-off exploratory behaviors while retaining nodes that have been reinforced through repeated interactions.

4 Experiments

4.1 Experimental Setup

4.1.1 Dataset. We evaluate on anonymized POI interaction logs from a large-scale Korean map service, comprising click, save, search, and review events. The evaluation window is 14 days, ending 2026-04-15. From the candidate population we sample users deterministically by hash order (seed 42) and keep those satisfying (i) at least one review event in the window, (ii) an alive target POI with a top-level category, and (iii) a large enough distractor pool in the same category. For each user, the most-recent review is the MCQ target and the preceding events build the memory. We report results at N=500 users; at this scale the Wilson 95% CI on aggregate accuracy is ±0.038, tight enough to detect the elevation-prompt accuracy gap reported in Section 4.3.

4.1.2 MCQ Evaluation Task. For each user we construct a 4-way multiple-choice question over their most-recent review's target POI. The three distractors are drawn from the same top-level category as the target and matched on review-count tier (±20% primary, with ±50% and ±100% fallbacks), excluding all POIs the user has previously interacted with. Each question is typed by what it probes:

- **T1 Recall** (n=361, 72.2%): the target's top-level category appears among the user's reviewed categories within the window; tests basic memory recall.
- **T2 Cross-signal** (n=33, 6.6%): the target's category was touched only through implicit signals (save/click) rather than review; tests cross-signal generalization.
- **T3 Evolution** (n=106, 21.2%): the target's category did not appear in any of the user's recorded interactions (review/save/click) within the window; tests whether the memory captures latent preferences that emerge at the test boundary.

The system observes only the constructed memory (not raw events). We report mean per-user accuracy (random = 0.25) and per-type accuracy across T1/T2/T3.

4.1.3 LLM Backbone. All LLM-mediated operations in this paper use Qwen3-235B-A22B-2507 [1], a 235B-parameter mixture-of-experts model with 22B activated parameters, served in-house via vLLM [2]. The Memory Agent (signal extraction, dynamic routing, forward update, and the merge / elevate / prune steps of the backward consolidation pass) and the persona-synthesis stage use the *instruct-2507* checkpoint. The MCQ evaluator uses the *thinking-2507* reasoning variant of the same model family. We use the same backbone across all baselines and ablation variants so that observed differences are attributable to memory design rather than model capability.

4.1.4 Baselines. We compare against four baselines spanning the design space:

- **NoMem** (No-Memory): LLM with only the four MCQ options and target category, with no user-specific context. Lower bound.
- **FullCtx** (Full-Context): All raw 14-day interaction events placed in the LLM context window. Quality upper bound, impractical at deployment scale.
- **Flat**: Mem0-style [5] flat list of memory entries with add, update, and delete operations. Tests the value of hierarchy.
- **FixedHier**: PersonaMem-v2-style [3] three-level tree with predefined fixed slots, no dynamic routing, and no backward consolidation. Tests the value of dynamic routing and backward maintenance.

The proposed **Ours** system uses dynamic routing, common trait elevation *with the concrete-noun guard from Section 3.5.2 as the production default*, three-axis preference separation, and the full backward consolidation pass.

4.1.5 Ablation Variants. To isolate the contribution of each proposed component, we evaluate seven variants of **Ours**. Five variants disable a single sub-step of the system, and two compare alternative implementations of the elevation step:

- **−routing**: Disable dynamic path creation; route only into pre-existing nodes.
- **−elevation**: Disable common-trait elevation entirely.
- **−backward**: Remove the entire backward consolidation pass (merge + elevate + prune).
- **−merge**: Disable the sibling-merge step in the backward pass.
- **−prune**: Disable stale-node pruning.
- **lift_destructive**: The concrete-noun guard is removed; the elevation prompt produces an abstracted parent and rewrites children to remove the elevated content, with no constraint on preserving concrete nouns. This is the naive elevation baseline against which the guard's contribution is measured.
- **lift_additive**: A non-destructive elevation variant in which the parent receives the shared trait but child notes are left intact (no rewriting). Tests whether structural rewriting itself is the issue.

The first five identify which structural component drives accuracy or footprint. The latter two, together with **Ours** (which uses the concrete-noun-guarded elevation), form a three-way comparison of elevation prompt formulations that we report as our main result.

4.2 Main Results

Table 1 (top block) reports MCQ accuracy and per-user memory footprint on N=500 users for the four baselines and the proposed

system. At this scale the Wilson 95% confidence interval on aggregate accuracy is ±0.038; accuracy differences smaller than this magnitude are not statistically resolved on Mean, though per-type CIs are looser (T1 ±0.046, T3 ±0.084, T2 ±0.16).

Table 1 yields five observations.

Hierarchy contributes accuracy. All memory-equipped systems beat the No-Memory floor (0.248) by a wide margin, so the MCQ is not solvable from category statistics alone. Among the deployable systems, the flat list (Flat, 0.672) trails FixedHier and Ours by about 0.09, well outside the CI. Hierarchy alone contributes accuracy, independent of other design choices.

The top three configurations are statistically tied. FullCtx (0.744), FixedHier (0.760), and Ours (0.758) sit within 0.016 of each other, inside the ±0.038 Wilson 95% CI. At this evaluation scale the three are not separable on aggregate accuracy. We therefore report Ours as *statistically indistinguishable* from both FixedHier (Δ=−0.002) and the FullCtx oracle (Δ=+0.014).

Footprint at parity. At this same accuracy level, Ours is the most compact deployable memory: 2,621 tokens per user, 28× smaller than FullCtx (∼73K) and 12% smaller than FixedHier (2,973). Flat is smaller still (1,210 tokens), but only by giving up 0.09 accuracy.

Per-type behavior. On T3 Evolution (queries about categories the user has not touched in the window), Ours and FixedHier are tied at 0.698, and both *exceed* the FullCtx oracle (0.632). The gap (0.066) lies just inside the per-type CI (±0.084), so we report it as a consistent direction rather than a statistically significant effect: structured memory appears to handle latent long-tail preferences better than raw context. On T2 Cross-signal, FixedHier leads numerically (0.606 vs. 0.515), but the n=33 sample gives a ±0.16 CI that absorbs the difference.

Three-axis compression and Pareto-optimality. The 12% total-token reduction vs. FixedHier appears on two finer axes simultaneously: Ours uses fewer notes per user (9.9 vs. 10.8) *and* shorter notes (264 vs. 275 tokens) at the same accuracy. We define the *Memory Efficiency Score*

$$\text{MES} = \frac{\text{Mean} - \text{Mean}_{\text{NoMem}}}{\log_{10}(\text{Tokens}/u)}, \tag{5}$$

subtracting the random-guess floor and discounting the memory budget on the log scale (the natural scale for context-window cost). Ours ranks first on MES (0.149; FixedHier 0.147, Flat 0.138, FullCtx 0.102, every ablation variant ≤ 0.145 in Table 1). Restricting to configurations reaching ≥ 95% of the FullCtx ceiling (≥ 0.707; Flat fails), Ours has the smallest footprint on every axis at once, a *Pareto-optimal* operating point: FixedHier and FullCtx fall inside Ours' ±0.038 CI band with 13% and 28× larger footprints.

4.3 Ablation Study

Table 1 (bottom block) reports the seven ablation variants of **Ours** on N=500 users, grouped by what they perturb: (a) forward routing, (b) a single backward sub-step, (c) the entire backward pass, and (d) the elevation prompt. Group (d) is our main result; the others identify the sources of accuracy and footprint.

4.3.1 Elevation prompt formulation drives accuracy (Group d). Accuracy moves monotonically across the three elevation variants (`lift_additive` 0.724 → `lift_destructive` 0.740 → Ours 0.758), with the same ranking on T1 Recall (0.765 → 0.776 → 0.798) and T3 Evolution (0.670 → 0.689 → 0.698); the guarded variant also has the smallest footprint of the three (2,621 tokens). Shorter notes *with* higher accuracy is not a trade-off but a sign that the guard removes noise: an abstract phrase like "ethnic cuisine" replacing "Korean, Chinese, Japanese" inflates the parent without adding discriminative information, yielding Ours' MES of 0.149 vs. 0.137–0.144 for the other elevation variants. On a single user's *Restaurants* subtree, destructive collapses concrete leaf words (*heukdwaeji*, *izakaya*, *hot pot*) into the parent's abstract trait, additive inflates the parent at unchanged leaves, and Ours' guard keeps those concrete leaves under an abstract parent.

4.3.2 Backward pass primarily controls memory size (Groups b, c). Every backward variant moves accuracy by no more than the ±0.038 CI (−merge and −elevation cost 0.032; −prune 0.014; the full −backward 0.024), but −backward inflates memory by 16.4%, roughly 4.3M extra tokens per day at our 10K-user daily-active scale. The two sub-steps compress along different axes: without merge, notes per user jump (11.3 vs. 9.9) while each shortens (241 vs. 264 tokens); without elevation, the count holds near reference (10.1) while each note grows (288 tokens). That is: *merge reduces note count; elevation reduces note length.*

4.3.3 Forward routing helps accuracy at no footprint cost (Group a). Disabling dynamic routing costs 0.022 accuracy (within CI) for only 38 saved tokens per user (1.4%): routing helps accuracy at near-zero footprint cost.

4.3.4 Per-type pattern matches the over-abstraction story. The per-type breakdown tracks the over-abstraction failure in Section 3.5.2: the guard's biggest gain is on T1 Recall, which depends on concrete category words appearing in the persona (`lift_destructive` 0.776 → Ours 0.798); T3 Evolution moves in the same direction (0.689 → 0.698). T2 Cross-signal is flat across the three elevation variants (≈ 0.515), inside the n=33 noise band. The guard itself adds zero LLM calls (one extra line in the existing elevation prompt, Eq. (1)), so the gain is obtained at no added inference cost.

5 Conclusion and Future Work

We presented a POI-centric hierarchical note-based memory combining dynamic routing, three-axis preference separation, and backward consolidation. Our main result at N=500: the elevation *prompt* is the primary design choice. Two naive variants underperform FixedHier (0.740 and 0.724 vs. 0.760), but a one-line concrete-noun guard recovers accuracy to 0.758, tied with FixedHier and the FullCtx oracle within the ±0.038 CI while running 28× smaller than FullCtx and 12% smaller than FixedHier on tokens. The backward pass primarily serves as a compression mechanism: removing it inflates memory by 16% at no measurable accuracy cost, and its sub-steps compress along independent axes (merge reduces count, elevation reduces length). For deployed POI memory, accuracy is set by elevation prompt design; structural backward operations function as cost–quality controls.

Table 1: MCQ accuracy and per-user memory size at N=500. *Top block*: baselines and reference points. *Bottom block*: ablations of Ours grouped by (a) forward routing, (b) one backward sub-step removed, (c) entire backward pass removed, (d) elevation prompt variant. Accuracy columns: aggregate Mean and per-type (T1 Recall, T2 Cross-signal, T3 Evolution). Size columns: notes per user, tokens per note, total tokens per user; MES is the Memory Efficiency Score (Eq. 5). Bold marks the best value within each block. Wilson 95% CI: Mean ±0.038; T1 ±0.046; T3 ±0.084; T2 ±0.16. † = non-deployable oracle.

	Accuracy				Memory size / user			
Method	Mean	T1	T2	T3	Notes	Tokens/note	Tokens	MES
Baselines and reference points								
NoMem (random floor)	0.248	0.241	0.303	0.255	0	—	0	—
FullCtx† (oracle)	0.744	0.800	0.485	0.632	—	—	~73,000	0.102
Flat [5]	0.672	0.701	0.485	0.632	49.9	24	**1,210**	0.138
FixedHier [3]	**0.760**	0.792	**0.606**	**0.698**	10.8	275	2,973	0.147
Ours (concrete-noun guard)	0.758	**0.798**	0.515	**0.698**	**9.9**	**264**	2,621	**0.149**
Ablations of Ours (compare each row to Ours above)								
(a) Forward routing								
−routing	0.736	0.762	0.515	**0.717**	9.8	265	2,583	0.143
(b) Backward pass — one sub-step removed								
−merge	0.726	0.756	**0.576**	0.670	11.3	**241**	2,722	0.139
−elevation	0.726	0.767	0.485	0.660	10.1	288	2,908	0.138
−prune	0.744	0.781	0.545	0.679	10.2	259	2,649	0.145
(c) Entire backward pass removed								
−backward (full)	0.734	0.770	0.515	0.679	11.2	273	3,052	0.140
(d) Elevation prompt variant — our main ablation result								
lift_additive	0.724	0.765	0.455	0.670	10.2	298	3,030	0.137
lift_destructive (no guard)	0.740	0.776	0.515	0.689	10.1	263	2,648	0.144

Three limitations remain. First, our tree does not link nodes across branches; A-Mem [6]'s cross-node activation could extend trait elevation toward multi-hop reasoning. Second, every memory operation relies on LLM judgment, with a 73K-token implicit extraction step dominating the forward pass; plausible 7–50× reductions include rule-based thresholds for the cheapest steps, distilling the agent [3], and clustering similar inputs before the LLM [19]. Third, prompts and the POI taxonomy are Korean; other languages require taxonomy adaptation and translation.

References

[1] A. Yang, A. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, G. Dong, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[3] B. Jiang, Y. Yuan, M. Shen, Z. Hao, Z. Xu, Z. Chen, Z. Liu, A. R. Vijjini, J. He, H. Yu, R. Poovendran, G. Wornell, L. Ungar, D. Roth, S. Chen, and C. J. Taylor. PersonaMem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. *arXiv preprint arXiv:2512.06688*, 2025.

[4] B. Jiang, Z. Hao, Y.-M. Cho, B. Li, Y. Yuan, S. Chen, L. Ungar, C. J. Taylor, and D. Roth. Know me, respond to me: Benchmarking LLMs for dynamic user profiling and personalized responses at scale. In *COLM*, 2025. arXiv:2504.14225.

[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[6] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. In *NeurIPS*, 2025. arXiv:2502.12110.

[7] S. Zhou and J. Han. A simple yet strong baseline for long-term conversational memory of LLM agents. *arXiv preprint arXiv:2511.17208*, 2025.

[8] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[9] N. Pollertlam and W. Kornsuwannawit. Beyond the context window: A cost-performance analysis of fact-based memory vs. long-context LLMs for persistent agents. *arXiv preprint arXiv:2603.04814*, 2026.

[10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the ACL*, 2024.

[11] T. Ge, X. Chan, X. Wang, D. Yu, H. Mi, and D. Yu. Scaling synthetic data creation with 1,000,000,000 personas. *arXiv preprint arXiv:2406.20094*, 2024.

[12] V. Samuel et al. PersonaGym: Evaluating persona agents and LLMs. In *Findings of EMNLP*, 2025. arXiv:2407.18416.

[13] J. Tan, L. Yang, Z. Liu, Z. Liu, R. Murthy, T. M. Awalgaonkar, J. Zhang, W. Yao, M. Zhu, S. Kokane, S. Savarese, H. Wang, C. Xiong, and S. Heinecke. PersonaBench: Evaluating AI models on understanding personal information through accessing (synthetic) private user data. In *Findings of ACL*, 2025. arXiv:2502.20616.

[14] M. J. Ryan, O. Shaikh, A. Bhagirath, D. Frees, W. Held, and D. Yang. SynthesizeMe! Inducing persona-guided prompts for personalized reward models in LLMs. In *ACL*, 2025. arXiv:2506.05598.

[15] Y. Zhang, Y. Zhang, Y. Wang, T. Chen, and H. Yin. ProEx: A unified framework leveraging large language model with profile extrapolation for recommendation. In *KDD*, 2026. arXiv:2512.00679.

[16] S. Feng, H. Lyu, C. Chen, and Y.-S. Ong. Where to move next: Zero-shot generalization of LLMs for next POI recommendation. *arXiv preprint arXiv:2404.01855*, 2024.

[17] C. Sun, K. Yang, R. G. Reddy, Y. R. Fung, H. P. Chan, K. Small, C. Zhai, and H. Ji. Persona-DB: Efficient large language model personalization for response prediction with collaborative data refinement. *arXiv preprint arXiv:2402.11060*, 2024.

[18] Z. Tan, J. Yan, I.-H. Hsu, R. Han, Z. Wang, L. T. Le, Y. Song, Y. Chen, H. Palangi, G. Lee, A. Iyer, T. Chen, H. Liu, C.-Y. Lee, and T. Pfister. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *ACL*, 2025. arXiv:2503.08026.

[19] Y. Shi, W. Xu, Z. Zhang, X. Zi, Q. Wu, and M. Xu. PersonaX: A recommendation agent oriented user modeling framework for long behavior sequence. In *Findings of ACL*, 2025. arXiv:2503.02398.

[20] V. Karpukhin et al. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

[21] G. Izacard et al. Unsupervised dense information retrieval with contrastive learning. *TMLR*, 2022.

[22] P. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[23] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[24] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[25] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

[26] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
[27] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*, 2024.
[28] W.-C. Kang and J. McAuley. Self-attentive sequential recommendation. In *ICDM*, 2018.
[29] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang. BERT4Rec: Sequential recommendation with bidirectional encoder representations from Transformer. In *CIKM*, 2019.
[30] J. Feng, Y. Li, C. Zhang, F. Sun, F. Meng, A. Guo, and D. Jin. DeepMove: Predicting human mobility with attentional recurrent networks. In *WWW*, 2018.
[31] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *RecSys*, 2022.
[32] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *RecSys*, 2023.
[33] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.
[34] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of AAAI*, 2024.
[35] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *NeurIPS*, 2024.
[36] A. Salemi, S. Mysore, M. Bendersky, and H. Zamani. LaMP: When large language models meet personalization. In *ACL*, 2024.
[37] Z. Tan, Q. Zeng, Y. Tian, Z. Liu, B. Yin, and M. Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning (OPPU). In *EMNLP*, 2024.
[38] S. Yang, J. Liu, and K. Zhao. GETNext: Trajectory flow map enhanced transformer for next POI recommendation. In *SIGIR*, 2022.
[39] Y. Luo, Q. Liu, and Z. Liu. STAN: Spatio-temporal attention network for next location recommendation. In *WWW*, 2021.
[40] J. Liao, S. Li, Z. Yang, J. Wu, Y. Yuan, X. Wang, and X. He. LLaRA: Large language-recommendation assistant. In *SIGIR*, 2024.
[41] W. Sun, L. Yan, X. Ma, S. Wang, P. Ren, Z. Chen, D. Yin, and Z. Ren. Is ChatGPT good at search? Investigating large language models as re-ranking agents. In *EMNLP*, 2023.
[42] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.
[43] L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. Shen, C. Qin, C. Zhu, H. Zhu, Q. Liu, H. Xiong, and E. Chen. A survey on large language models for recommendation. *World Wide Web*, 27, Article 60, 2024.
[44] Y. Zhuang, H. Sun, Y. Yu, R. Qiang, Q. Wang, C. Zhang, and B. Dai. HYDRA: Model factorization framework for black-box LLM personalization. In *NeurIPS*, 2024.
[45] Y. Wang, Z. Jiang, Z. Chen, F. Yang, Y. Zhou, E. Cho, X. Fan, Y. Lu, X. Huang, and Y. Yang. RecMind: Large language model powered agent for recommendation. In *NAACL Findings*, 2024.
[46] G. Kamradt. Needle in a haystack—pressure testing LLMs. GitHub repository, `gkamradt/LLMTest_NeedleInAHaystack`, 2023.
[47] J. Shen, Z. Shen, C. Xiong, C. Wang, K. Wang, and J. Han. TaxoExpan: Self-supervised taxonomy expansion with position-enhanced graph neural network. In *WWW*, 2020.